

第五次实验

1. JoCoCo 插件配置

增加 pom.xml 文件中 jacoco-maven-plugin 插件配置, 并补充所需的 Mockito 依赖。



```
63 </dependency>  
64  
65 <!-- Mockito 用于 mock 依赖对象 -->  
66 <dependency>  
67   <groupId>org.mockito</groupId>  
68   <artifactId>mockito-core</artifactId>  
69   <version>2.23.4</version>  
70   <scope>test</scope>  
  </dependency>
```

```

88      <!-- JaCoCo 覆盖率插件 -->
89      <plugin>
90          <groupId>org.jacoco</groupId>
91          <artifactId>jacoco-maven-plugin</artifactId>
92          <version>0.8.8</version>
93          <executions>
94              <execution>
95                  <id>prepare-agent</id>
96                  <goals>
97                      <goal>prepare-agent</goal>
98                  </goals>
99              </execution>
100             <execution>
101                 <id>report</id>
102                 <phase>test</phase>
103                 <goals>
104                     <goal>report</goal>
105                 </goals>
106             </execution>
107         </executions>
108     </plugin>
109 </plugins>

```

2. 选用所提供 studentmanager-master 项目的课程管理模块 (CourseController)

负责该模块的 HTTP 请求处理共有 5 个方法。

方法名	HTTP 方式
courseList()	GET
getClazzList(...)	POST
addCorse(Course)	POST
editCourse(Course)	POST
deleteCourse(Data)	POST

```

@GetMapping("/course_list") no usages
public String courseList(){
    return "course/courseList";
}

```

```

@PostMapping("/getCouseList") no usages
@ResponseBody
public Object getClazzList(@RequestParam(value = "page", defaultValue = "1")Integer page,
    @RequestParam(value = "rows", defaultValue = "100")Integer rows,
    String name,
    @RequestParam(value = "teacherid", defaultValue = "0")String teacherid ,String from){
    Map<String, Object> paramMap = new HashMap();
    paramMap.put("pageno", page);
    paramMap.put("pagesize", rows);
    if(!StringUtils.isEmpty(name)) paramMap.put("name", name);
    if(!teacherid.equals("0")) paramMap.put("teacherId", teacherid);
    PageBean<Course> pageBean = courseService.queryPage(paramMap);
    if(!StringUtils.isEmpty(from) && from.equals("combox")){
        return pageBean.getDatas();
    }else{
        Map<String, Object> result = new HashMap();
        result.put("total", pageBean.getTotalSize());
        result.put("rows", pageBean.getDatas());
        return result;
    }
}

```

```
@PostMapping("/addCourse") no usages
@ResponseBody
public AjaxResult addCourse(Course course){
    AjaxResult ajaxResult = new AjaxResult();
    try {
        int count = courseService.addCourse(course);
        if(count > 0){
            ajaxResult.setSuccess(true);
            ajaxResult.setMessage("添加成功");
        }else{
            ajaxResult.setSuccess(false);
            ajaxResult.setMessage("添加失败");
        }
    }catch (Exception e){
        e.printStackTrace();
        ajaxResult.setSuccess(false);
        ajaxResult.setMessage("添加失败");
    }
    return ajaxResult;
}
```

```
@PostMapping("/editCourse") no usages
@ResponseBody
public AjaxResult editCourse(Course course){
    AjaxResult ajaxResult = new AjaxResult();
    try {
        int count = courseService.editCourse(course);
        if(count > 0){
            ajaxResult.setSuccess(true);
            ajaxResult.setMessage("修改成功");
        }else{
            ajaxResult.setSuccess(false);
            ajaxResult.setMessage("修改失败");
        }
    } catch (Exception e){
        e.printStackTrace();
        ajaxResult.setSuccess(false);
        ajaxResult.setMessage("修改失败");
    }
    return ajaxResult;
}
```



```

@PostMapping("/deleteCourse") no usages
@ResponseBody
public AjaxResult deleteCourse(Data data){
    AjaxResult ajaxResult = new AjaxResult();
    try {
        int count = courseService.deleteCourse(data.getIds());
        if(count > 0){
            ajaxResult.setSuccess(true);
            ajaxResult.setMessage("删除成功");
        }else{
            ajaxResult.setSuccess(false);
            ajaxResult.setMessage("删除失败");
        }
    }catch (Exception e){
        e.printStackTrace();
        ajaxResult.setSuccess(false);
        ajaxResult.setMessage("删除失败,该班级存在老师或学生");
    }
    return ajaxResult;
}

```

3. 单元测试类设计

使用@RunWith(SpringRunner.class)、@InjectMocks、@Mock 完成依赖注入。

```

@RunWith(SpringRunner.class)
public class CourseControllerTest {

    /** 被测 Controller, 由Mockito 注入 Mock 的 Service */
    @InjectMocks 1 usage
    private CourseController courseController;

    /** Mock 掉 CourseService, 避免连接真实数据库 */
    @Mock 14 usages
    private CourseService courseService;

```

在@Before 方法中构建 MockMvc, 不启动 Spring 容器。

```

/**
 * 每个测试方法执行前, 初始化 Mock 注解并构建 MockMvc
 */
@Before
public void setUp() {
    MockitoAnnotations.initMocks( testClass: this);
    mockMvc = MockMvcBuilders.standaloneSetup(courseController).build();
}

```

设计 15 个测试方法, 覆盖全部 5 个方法的所有分支路径。

```

/**
 * 测试 courseList(): 访问 GET /course/course_list
 * 预期: 返回逻辑视图名 "course/courseList", HTTP 200
 */
@Test
public void testCourseList_returnsViewName() throws Exception {
    mockMvc.perform(get( urlTemplate: "/course/course_list"))
        .andExpect(status().isOk())
        .andExpect(view().name( expectedViewName: "course/courseList"));
}

```

```

/**
 * 测试getClazzList(): 普通分页查询 (无from=combox)
 * 分支: name 为空、teacherid=0、from 为空
 * 预期: JSON 包含total 和rows 字段
 */
@Test
public void testGetCourseList_normalResponse() throws Exception {
    // 构造 mock 返回的分页数据
    List<Course> courseList = new ArrayList<>();
    Course c1 = new Course();
    c1.setId(1);
    c1.setName("Java程序设计");
    courseList.add(c1);

    PageBean<Course> pageBean = new PageBean<>(pageno: 1, pagesize: 10);
    pageBean.setDatas(courseList);
    pageBean.setTotalsize(1);

    when(courseService.queryPage(anyMap())).thenReturn(pageBean);

    mockMvc.perform(post(urlTemplate: "/course/getCourseList")
        .param(name: "page", ...values: "1")
        .param(name: "rows", ...values: "10")
        .param(name: "teacherid", ...values: "0"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$.total").value(expectedValue: 1))
        .andExpect(jsonPath(expression: "$.rows").isArray());
}

```



```
/**
 * 测试getClazzList(): 带name过滤参数 (name 非空分支)
 * 预期: paramMap 中包含name 键
 */
@Test
public void testGetCourseList_withNameFilter() throws Exception {
    PageBean<Course> pageBean = new PageBean<> (pageno: 1, pagesize: 10);
    pageBean.setDatas(new ArrayList<>());
    pageBean.setTotalsize(0);

    when(courseService.queryPage(anyMap())).thenReturn(pageBean);

    mockMvc.perform(post( urlTemplate: "/course/getCourseList")
        .param( name: "page", ...values: "1")
        .param( name: "rows", ...values: "10")
        .param( name: "name", ...values: "数学")
        .param( name: "teacherid", ...values: "0"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$.total").value( expectedValue: 0));
}
```

```
/**
 * 测试getClazzList(): 带teacherid 非0 过滤 (teacherId 分支)
 * 预期: 正常返回 rows
 */
@Test
public void testGetCourseList_withTeacherIdFilter() throws Exception {
    PageBean<Course> pageBean = new PageBean<>(pageno: 1, pagesize: 10);
    pageBean.setDatas(new ArrayList<>());
    pageBean.setTotalsize(0);

    when(courseService.queryPage(anyMap())).thenReturn(pageBean);

    mockMvc.perform(post( urlTemplate: "/course/getCourseList")
        .param( name: "page", ...values: "1")
        .param( name: "rows", ...values: "10")
        .param( name: "teacherid", ...values: "5"))
        .andDo(print())
        .andExpect(status().isOk());
}
```

```
/**
 * 测试getClazzList(): from=combox 分支
 * 预期: 直接返回 Course 列表数组, 而非包含total/rows的对象
 */
@Test
public void testGetCourseList_comboxResponse() throws Exception {
    List<Course> courseList = new ArrayList<>();
    Course c1 = new Course();
    c1.setId(2);
    c1.setName("数据结构");
    courseList.add(c1);

    PageBean<Course> pageBean = new PageBean<>(pageno: 1, pagesize: 100);
    pageBean.setDatas(courseList);
    pageBean.setTotalsize(1);

    when(courseService.queryPage(anyMap())).thenReturn(pageBean);

    mockMvc.perform(post(urlTemplate: "/course/getCourseList")
        .param(name: "page", ...values: "1")
        .param(name: "rows", ...values: "100")
        .param(name: "teacherid", ...values: "0")
        .param(name: "from", ...values: "combox"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$[0].name").value(expectedValue: "数据结构"));
}
```



```
* 测试getClazzList(): from 非空但不等于"combox"的分支
* 覆盖from 非空 && from != "combox" → 走else分支返回total+rows的场景
* 这是"!StringUtils.isEmpty(from) && from.equals("combox")"中
* 第二个条件为false时的路径, 也是Branch 100% 所缺少的最后一个分支
* 预期: 返回包含total 和rows 的JSON 对象
*/
@Test
public void testGetCourseList_fromNotCombox() throws Exception {
    List<Course> courseList = new ArrayList<>();
    Course c1 = new Course();
    c1.setId(3);
    c1.setName("操作系统");
    courseList.add(c1);

    PageBean<Course> pageBean = new PageBean<>(pageno: 1, pagesize: 10);
    pageBean.setDatas(courseList);
    pageBean.setTotalsize(1);

    when(courseService.queryPage(anyMap())).thenReturn(pageBean);

    // from 传 "list" (非空, 但不是 "combox"), 走 else 分支
    mockMvc.perform(post(urlTemplate: "/course/getCourseList")
        .param(name: "page", ...values: "1")
        .param(name: "rows", ...values: "10")
        .param(name: "teacherid", ...values: "0")
        .param(name: "from", ...values: "list"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$.total").value( expectedValue: 1))
        .andExpect(jsonPath(expression: "$.rows").isArray());
}
```



```

/**
 * 测试addCourse(): 添加成功 (count > 0)
 * 预期: success=true, message="添加成功"
 */
@Test
public void testAddCourse_success() throws Exception {
    when(courseService.addCourse(any(Course.class))).thenReturn( t: 1);

    mockMvc.perform(post( urlTemplate: "/course/addCourse")
        .param( name: "name", ...values: "软件工程")
        .param( name: "teacherId", ...values: "3")
        .param( name: "courseDate", ...values: "周三")
        .param( name: "maxNum", ...values: "60")
        .param( name: "info", ...values: "软件工程课程简介"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$.success").value( expectedValue: true))
        .andExpect(jsonPath( expression: "$.message").value( expectedValue: "添加成功"));
}

```

```

/**
 * 测试addCourse(): 添加失败 (count <= 0)
 * 预期: success=false, message="添加失败"
 */
@Test
public void testAddCourse_fail() throws Exception {
    when(courseService.addCourse(any(Course.class))).thenReturn( t: 0);

    mockMvc.perform(post( urlTemplate: "/course/addCourse")
        .param( name: "name", ...values: "失败课程"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$.success").value( expectedValue: false))
        .andExpect(jsonPath( expression: "$.message").value( expectedValue: "添加失败"));
}

```



```

/**
 * 测试addCourse(): Service 抛出异常 (异常分支)
 * 预期: 捕获异常后返回 success=false, message="添加失败"
 */
@Test
public void testAddCourse_exception() throws Exception {
    when(courseService.addCourse(any(Course.class)))
        .thenReturn(new RuntimeException("DB connection error"));

    mockMvc.perform(post(urlTemplate: "/course/addCourse")
        .param(name: "name", ...values: "异常课程"))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$.success").value( expectedValue: false))
        .andExpect(jsonPath(expression: "$.message").value( expectedValue: "添加失败"));
}

```

```

/**
 * 测试editCourse(): 修改成功 (count > 0)
 * 预期: success=true, message="修改成功"
 */
@Test
public void testEditCourse_success() throws Exception {
    when(courseService.editCourse(any(Course.class))).thenReturn( t 1);

    mockMvc.perform(post(urlTemplate: "/course/editCourse")
        .param(name: "id", ...values: "1")
        .param(name: "name", ...values: "修改后课程名")
        .param(name: "teacherId", ...values: "2"))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$.success").value( expectedValue: true))
        .andExpect(jsonPath(expression: "$.message").value( expectedValue: "修改成功"));
}

```

```

/**
 * 测试editCourse(): 修改失败 (count <= 0)
 * 预期: success=false, message="修改失败"
 */
@Test
public void testEditCourse_fail() throws Exception {
    when(courseService.editCourse(any(Course.class))).thenReturn(0);

    mockMvc.perform(post(urlTemplate: "/course/editCourse")
        .param(name: "id", ...values: "999"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$.success").value(expectedValue: false))
        .andExpect(jsonPath(expression: "$.message").value(expectedValue: "修改失败"));
}

```

```

/**
 * 测试editCourse(): Service 抛出异常 (异常分支)
 * 预期: success=false, message="修改失败"
 */
@Test
public void testEditCourse_exception() throws Exception {
    when(courseService.editCourse(any(Course.class)))
        .thenThrow(new RuntimeException("update failed"));

    mockMvc.perform(post(urlTemplate: "/course/editCourse")
        .param(name: "id", ...values: "1"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$.success").value(expectedValue: false))
        .andExpect(jsonPath(expression: "$.message").value(expectedValue: "修改失败"));
}

```

```

/**
 * 测试 deleteCourse(): 删除成功 (count > 0)
 * 预期: success=true, message="删除成功"
 */
@Test
public void testDeleteCourse_success() throws Exception {
    when(courseService.deleteCourse(anyList())).thenReturn( t: 1);

    mockMvc.perform(post( urlTemplate: "/course/deleteCourse")
        .param( name: "ids", ...values: "1", "2"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$.success").value( expectedValue: true))
        .andExpect(jsonPath( expression: "$.message").value( expectedValue: "删除成功"));
}

```

```

/**
 * 测试 deleteCourse(): 删除失败 (count <= 0)
 * 预期: success=false, message="删除失败"
 */
@Test
public void testDeleteCourse_fail() throws Exception {
    when(courseService.deleteCourse(anyList())).thenReturn( t: 0);

    mockMvc.perform(post( urlTemplate: "/course/deleteCourse")
        .param( name: "ids", ...values: "999"))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$.success").value( expectedValue: false))
        .andExpect(jsonPath( expression: "$.message").value( expectedValue: "删除失败"));
}

```


Coverage CourseControllerTest				
Element ^	Class, %	Method, %	Line, %	Branch, %
com.wdd.studentmanager.controller	10% (1/10)	9% (5/53)	6% (46/703)	5% (14/247)
AdminController	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
AttendanceController	0% (0/1)	0% (0/5)	0% (0/47)	0% (0/22)
ClazzController	0% (0/1)	0% (0/5)	0% (0/52)	0% (0/16)
CourseController	100% (1/1)	100% (5/5)	100% (46/46)	100% (14/14)
LeaveController	0% (0/1)	0% (0/6)	0% (0/56)	0% (0/14)
ScoreController	0% (0/1)	0% (0/9)	0% (0/166)	0% (0/58)
SelectedCourseController	0% (0/1)	0% (0/4)	0% (0/40)	0% (0/18)
StudentController	0% (0/1)	0% (0/5)	0% (0/87)	0% (0/34)
SystemController	0% (0/1)	0% (0/8)	0% (0/128)	0% (0/41)
TeacherController	0% (0/1)	0% (0/5)	0% (0/80)	0% (0/30)